COL761: Data Mining
Assignment 1
Frequent Subgraph Mining: Comparative Analysis

Jayesh Nandu Chaudhari (2025AIB3002)
Ankur Kumar (2025AIB2557)
Meghender Pal (2025AIB2563)
Indian Institute of Technology Delhi

1. Introduction

This report presents an empirical comparison of three frequent subgraph mining algorithms: gSpan, FSG (PAFI), and Gaston. The algorithms were evaluated on the Yeast dataset across five minimum support thresholds (5%, 10%, 25%, 50%, and 95%) to analyze their performance characteristics and scalability.

1.1. Hardware Details

All experiments were conducted on a workstation running Ubuntu 20.04.6 LTS. The system configuration is as follows:

- **Processor:** 12th Gen Intel® Core™ i9-12900 (24 cores)
- **Memory:** 31.0 GiB
- **Graphics:** Mesa Intel® Graphics (ADL-S GT1)
- **Storage Capacity:** 2.0 TB
- **GNOME Version:** 3.36.8
- **Windowing System:** X11

This hardware configuration ensures reliable and consistent performance measurements across all algorithmic evaluations.

2. Experimental Setup

2.1. Algorithms

- **gSpan** [1]: A depth-first search algorithm using DFS codes for canonical representation
- **FSG** [2]: A breadth-first approach based on candidate generation
- **Gaston** [3]: Employs a hybrid strategy with separate path and tree mining

2.2. Dataset

The Yeast dataset was used, containing molecular structures represented as labeled graphs with nodes representing atoms and edges representing chemical bonds.

2.3. Support Thresholds

Experiments were conducted at minimum support values of 5%, 10%, 25%, 50%, and 95% to observe algorithmic behavior across different frequency requirements.

3. Results

3.1. Runtime Performance

Table 1 summarizes the execution times for all three algorithms across different support thresholds.

Table 1: Runtime comparison (in seconds)

Support (%)	gSpan	FSG	Gaston
5	1327.62	1512.99	54.84
10	369.02	504.07	19.33
25	87.89	144.23	6.47
50	31.64	50.65	3.12
95	1.72	9.92	0.40

3.2. Performance Visualization

Figure 1 illustrates the runtime trends across support thresholds. The exponential growth in runtime as support decreases is clearly visible for all algorithms.

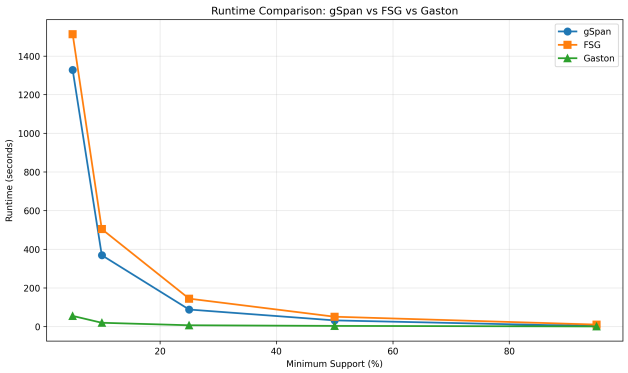

Figure 1: Runtime comparison of gSpan, FSG, and Gaston

4. Analysis and Observations

4.1. Overall Performance

Gaston consistently outperforms both gSpan and FSG across all support thresholds, achieving speedups of 10-24× over gSpan and 13-28× over FSG at lower support values.

4.2. Growth Rate Analysis

All three algorithms exhibit exponential runtime growth as minimum support decreases. This is expected because lower support thresholds result in:

- Larger candidate sets requiring enumeration
- More frequent subgraphs to discover
- Increased subgraph isomorphism tests

The runtime ratio between 5% and 95% support is approximately 773× for gSpan, 152× for FSG, and 137× for Gaston, indicating that gSpan's depth-first approach is most sensitive to support threshold changes.

4.3. Algorithmic Differences

Gaston's Superiority: Gaston's exceptional performance stems from its hybrid mining strategy [3]. It separately mines paths, trees, and cyclic graphs, exploiting the computational simplicity of path and tree enumeration. By deferring expensive cyclic graph mining until necessary, Gaston minimizes subgraph isomorphism tests.

gSpan vs. FSG: gSpan generally outperforms FSG, particularly at higher support thresholds (50% and 95%). This advantage comes from:

- DFS code-based canonical representation eliminating duplicate candidates
- Depth-first exploration enabling early pruning
- Rightmost extension strategy reducing search space

FSG's Apriori-like breadth-first approach generates and tests candidates level-by-level, leading to higher memory overhead and more redundant isomorphism checks [2].

4.4. Low Support Behavior

At 5% support, FSG becomes the slowest (1513s), exceeding gSpan (1328s) by 14%. This is because FSG must maintain and test large candidate sets at each level in memory, while gSpan's DFS approach processes candidates incrementally.

4.5. High Support Efficiency

At 95% support, the performance gap narrows but remains significant. FSG takes 9.92s compared to gSpan's 1.72s and Gaston's 0.40s. The overhead of FSG's candidate generation becomes more pronounced even when few patterns are frequent.

5. Conclusion

This empirical study demonstrates clear performance distinctions among frequent subgraph mining algorithms. Gaston emerges as the most efficient approach, leveraging structural decomposition to achieve superior scalability. gSpan's depth-first strategy with canonical forms provides better performance than FSG's breadth-first Apriori-based method, particularly as support decreases. The choice of algorithm should consider the expected support threshold and graph characteristics, with Gaston being preferred for general-purpose applications requiring robust performance across varying support levels.

References

[1] X. Yan and J. Han, "gSpan: Graph-Based Substructure Pattern Mining," in *Proceedings of the 2002 IEEE International Conference on Data Mining (ICDM)*, 2002, pp. 721–724.

[2] M. Kuramochi and G. Karypis, "Frequent Subgraph Discovery," in *Proceedings of the 2001 IEEE International Conference on Data Mining (ICDM)*, 2001, pp. 313–320.

[3] S. Nijssen and J. N. Kok, "A Quickstart in Frequent Structure Mining Can Make a Difference," in *Proceedings of the 10th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2004, pp. 647–652.

[4] Anthropic, "Claude," `https://www.anthropic.com/claude`, 2024. [AI assistant used for code development and debugging assistance].